

Written 2026-07-25. Companion to `docs/NEURAL-ARCHITECTURE.md` (what to build) and `docs/LITERATURE-v2.md` (the algorithm canon). This document states what game Champions actually is, what the correct solution concept therefore is, and which of our planned components are theoretically sound versus theoretically wrong.

It exists because one of our planned components **is provably wrong**, and that is worth writing down before it gets built rather than after.

Champions VGC is a two-player zero-sum game that is **simultaneously** three hard things at once:

property	consequence
simultaneous-move	both players commit each turn without seeing the other's choice
imperfect-information	sets, items, spreads and the unbrought two are hidden
stochastic	damage rolls, crits, accuracy, speed ties

Most of the game-AI canon handles one or two of these. Chess has none of them. Poker is imperfect-information and stochastic but **sequential** — players act in turn. Champions is the harder intersection, and the literature that matters is the part addressing simultaneous moves specifically.

Our own measurement of the branching factor (`engine/mew_farm.js`, 244 real decision points): **~100 legal options per side per turn, ~10,000 joint per turn, ~10⁶⁰ for a full game**. Exhaustive search is not merely impractical, it is absurd. Everything below is about what to do instead.

This is the central theoretical point and it invalidates a plan we had already sketched.

In a simultaneous-move game, any deterministic policy is exploitable. If your opponent knows you will always pick the highest-value action, they pick the counter to it. The equilibrium concept is a **mixed strategy** — a probability distribution over actions — exactly as in rock-paper-scissors.

The literature is unambiguous. Standard MCTS assumes one active player per node, which is simply the wrong model for simultaneous moves; it **converges to a pure strategy and is therefore exploitable** (Lanctot et al., Bosansky et al.). The fix is to treat each decision point as a **matrix game** and solve it — or, since an exact Nash per node means a linear program per node, to use a regret- minimising procedure that converges to one:

- **Decoupled UCB** — cheap, but still converges to a pure strategy; least principled.
- **Regret matching / EXP3** — Hannan-consistent; with self-play the average policy converges to an $O(D^2 \cdot \epsilon)$ -Nash equilibrium.
- **Online Outcome Sampling (OOS)** — derived from MCCFR, **provably converges to Nash** in this class.

Reported result: regret matching and OOS perform best, and **all variants are less exploitable than UCT**.

Earlier in this project we sketched depth-1 exhaustive search: enumerate all ~100 actions, evaluate each with a value net, pick the best. **That is the exploitable pure-strategy algorithm.** It is fine as a strength baseline and fine for *explaining* a position to a human — "this move was worth +14 points" — but it must not be presented as optimal play, and a good opponent who models it will beat it.

The correction is small in code and large in principle: at each decision point, build the payoff matrix over the joint action space using the value net at the leaves, then **solve the matrix game with regret matching** and play the resulting mixed strategy. Same evaluator, same enumeration, one extra step.

The convergence guarantees above are proven for **perfect-information** simultaneous-move games. Champions is imperfect-information as well, which puts it formally in the harder class that CFR and its descendants address. Regret matching at the node is therefore a **principled heuristic** here, not a proof of optimality. Saying otherwise would be overclaiming, and this project has done enough of that already.

Three components, each independently checkable:

- 1. A belief state** — a distribution over the opponent's hidden set, conditioned on what they have revealed. This is the information-set representation. *Status: implemented* in `engine/state_encoder.py` (`belief_top_p`, `belief_entropy`, `prior_coverage`), computed against measured usage priors. Verified rising over a game in 93% of games.
- 2. A learned leaf evaluator** — depth-limited search cannot reach terminal states, so it needs a value function at the horizon. This is DeepStack's continual re-solving shape. *Status: implemented* as PORY (`engine/pory_nn.py`), currently 0.5473 log-loss against a 0.5748 material baseline.
- 3. A mixed-strategy solver at each node** — regret matching over the joint action matrix. *Status: NOT built, and the thing section 2 says we must build instead of argmax.*

Damage rolls must be handled **analytically, not by enumeration**: 16 rolls per damaging move compounds into the 10^{60} . A closed-form KO probability collapses that entire dimension, and we already have the validated damage engine for it (31/31 within 2% of the official simulator).

system	our relationship to it
VGC-Bench (AAMAS 2026)	same format family. PPO + Transformer. Beat a pro on one team ; degrades beyond 1-3 teams
Metamon (RLC 2025)	singles, older gens. Offline RL at scale. Self-play data moved GXE from 41-58% to 64-80%
PokéAgent Challenge (NeurIPS 2025)	100+ teams, 20M+ trajectories. Concludes Pokémon is an unsolved benchmark , with "considerable gaps" between LLM, RL, and elite human play
DeepStack / ReBeL	the search shape we borrow: depth-limited, learned leaf values, re-solve the current subgame
SM-MCTS literature	the correction in section 2

The sobering datum: three independent research groups, with far more compute than this project has, all report the general problem unsolved. PokéAgent says so explicitly. That is not a reason to stop — it is a reason to be precise about which claim we are making.

Not "we solved VGC". The defensible claim, given everything above:

*For a **fixed team**, against the **measured metagame**, ABRA estimates win probability from a board state better than material alone, and can quantify how much a specific decision changed it.*

Every clause is falsifiable and every clause is currently measurable:

- *fixed team* — VGC-Bench degrades beyond 1-3 teams; our pool is 1,644. Scoping to one team is where the literature says results are actually achievable.
- *measured metagame* — priors come from measured usage, not invention.
- *better than material alone* — the B2 baseline, already implemented as the bar to clear.
- *quantify a decision* — advantage, $A(s,a) = Q(s,a) - V(s)$, computable exactly at depth 1.

That last clause is the product. Not "who wins", which we measured to be near-impossible from team sheets, but "**what did that turn cost you**", which is answerable and is what a player wants.

Written before the measurements, so they can be checked rather than rationalised:

1. **A value net trained only on self-play will transfer poorly to human games** — beating B2 by less on ladder than it does on self-play. The prior policy never reads the board, so it learns the format's physics, not a real game's pressure.
2. **History and belief features will help more on self-play than on ladder.** Ladder games are median 6 turns with 23.8% forfeits; there is almost no history to read. Self-play is median 10 with none. Measured on ladder, these features bought ~ 0.002 — consistent with "nothing to read".
3. **Deeper networks will keep not helping** until the representation changes. Measured: nonlinearity buys 0.0030, representation buys 0.0102.
4. **Argmax depth-1 search will be beatable by an opponent that models it**, per section 2.
Testable directly: play argmax against a regret-matching version of itself and measure exploitability.

If 1 and 2 both come out false, the self-play path is stronger than argued here and the 1M-game run is justified immediately.

- Lanctot, Lisý, Winands. *Monte Carlo Tree Search in Simultaneous Move Games with Applications to Goofspiel*. — <https://mlanctot.info/files/papers/wcg13-smmcts.pdf>
- Bošanský, Lisý, Lanctot, Čermák, Winands. *Algorithms for Computing Strategies in Two-Player Simultaneous Move Games*. — <https://dke.maastrichtuniversity.nl/m.winands/documents/sm-journal.pdf>
- *Convergence of Monte Carlo Tree Search in Simultaneous Move Games*. arXiv:1310.8613 — <https://arxiv.org/pdf/1310.8613>
- Karten et al. *The PokéAgent Challenge: Competitive and Long-Context Learning at Scale*. NeurIPS
- arXiv:2603.15563 — <https://arxiv.org/abs/2603.15563>
- Angliss, Cui, Hu, Rahman, Stone. *VGC-Bench*. AAMAS 2026. arXiv:2506.10326
- Grigsby et al. *Human-Level Competitive Pokémon via Scalable Offline RL with Transformers*. RLC
- arXiv:2504.04395